

ENABLE DYNAMIC CONFIGURATION AND FEATURE FLAGS

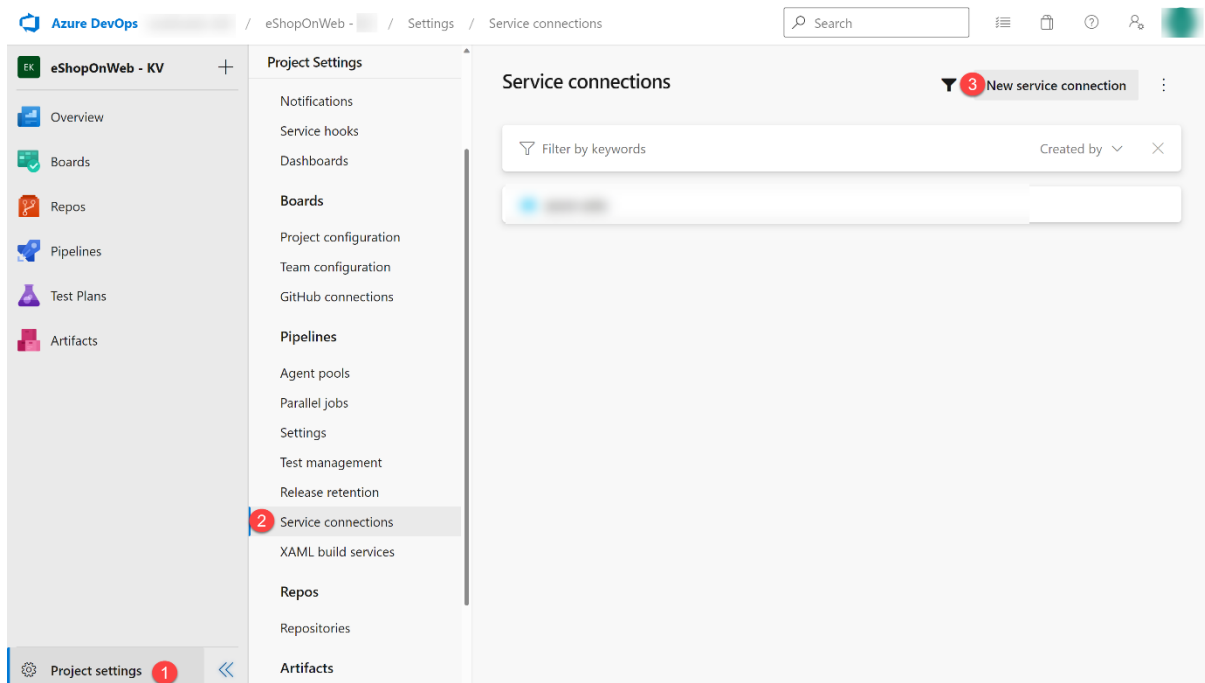
Create a service connection to access Azure resources

You will need to create a service connection in Azure DevOps which will allow you to deploy and access resources in your Azure subscription.

1. Start a web browser, navigate to the Azure DevOps portal <https://aex.dev.azure.com>.
2. Sign in to the Azure DevOps organization with the **Username** and **TAP** present on the **Resources** tab.

Note: If this is the first time you are signing in to the Azure DevOps organization, you will be prompted create your profile and accept the terms of service, and then select **Continue**.

3. Open **eShopOnWeb** project, and select **Project settings** in the bottom left corner of the portal.
4. Select the **Service connections** under Pipelines, and then select **Create service connection** button.



5. On the **New service connection** blade, select **Azure Resource Manager** and **Next** (you may need to scroll down).
6. Select **App registration (automatic)** from the **Identity type** dropdown.

7. Select **Workload Identity federation** and **Subscription** under the **Scope level**.

Note: You can also use **App registration or managed identity (manual)** if you prefer to manually configure the service connection. Follow the steps in the [Azure DevOps documentation](#) to create the service connection manually.

8. Fill in the empty fields using the information:
 - **Subscription:** Select your Azure subscription.
 - **Resource group:** Select the resource group AZ400-RG1.
 - **Service connection name:** Type **azure subs**. This name will be referenced in YAML pipelines to access your Azure subscription.
9. Make sure the **Grant access permission to all pipelines** option is unchecked and select **Save**.

Important: The **Grant access permission to all pipelines** option is not recommended for production environments. It is only used in this lab to simplify the configuration of the pipeline.

Note: If you see an error message indicating you don't have the necessary permissions to create a service connection, try again, or configure the service connection manually.

Enable Dynamic Configuration and Feature Flags

You will learn how to use Azure App Configuration to manage application settings and feature flags centrally. You'll explore how modern cloud applications can benefit from centralized configuration management and dynamic feature toggling without requiring application redeployment.

This lab takes approximately **45** minutes to complete.

Before you start

You need:

- **Microsoft Edge** or an [Azure DevOps supported browser](#)
- **Azure subscription:** You need an active Azure subscription or can create a new one
- **Azure DevOps organization:** Create one at [Create an organization or project collection](#) if you don't have one
- **Account permissions:** You need a Microsoft account or Microsoft Entra account with Contributor or Owner role in the Azure subscription

- For details, see [List Azure role assignments using the Azure portal](#) and [View and assign administrator roles in Azure Active Directory](#)

About Azure App Configuration

[Azure App Configuration](#) provides a service to manage application settings and feature flags centrally. Modern programs, especially those running in a cloud, generally have many distributed components. Spreading configuration settings across these components can lead to hard-to-troubleshoot errors during application deployment. Use App Configuration to store all the settings for your application and secure their access in one place.

Key benefits include:

- **Centralized management** of application settings and feature flags
- **Dynamic configuration** changes without application restart
- **Feature flag management** for controlled feature rollouts
- **Point-in-time configuration** snapshots for rollback scenarios
- **Integration** with Azure Key Vault for sensitive settings

Create and configure the team project

In this **Cloudslice** lab, please skip this task.

First, you'll create an Azure DevOps project for this lab.

1. In your browser, open your Azure DevOps organization
2. Select **New Project**
3. Give your project the name **eShopOnWeb**
4. Choose **Scrum** on the **Work Item process** dropdown
5. Select **Create**

Import the eShopOnWeb Git Repository

In this **Cloudslice** lab, please skip this task.

Next, you'll import the sample repository that contains the application code.

1. In your Azure DevOps organization, open the **eShopOnWeb** project
2. Select **Repos > Files**
3. Select **Import**

4. In the **Import a Git Repository** window, paste this URL: <https://github.com/MicrosoftLearning/eShopOnWeb.git>
5. Select **Import**

The repository is organized this way:

- **.ado** folder contains Azure DevOps YAML pipelines
 - **src** folder contains the .NET 8 website used in the lab scenarios
6. Go to **Repos > Branches**
 7. Hover on the **main** branch then select the ellipsis on the right
 8. Select **Set as default branch**

Import and run CI/CD Pipelines

You'll import CI/CD pipelines to build and deploy the eShopOnWeb application. The CI pipeline is prepared to build the application and run tests. The CD pipeline will deploy the application to an Azure Web App.

Import and run the CI pipeline

Let's start by importing the CI pipeline named [eshoponweb-ci.yml](#).

1. Go to **Pipelines > Pipelines**
2. Select **Create Pipeline** (if there are no pipelines) or **New pipeline** (if there are already created pipelines)
3. Select **Azure Repos Git (Yaml)**
4. Select the **eShopOnWeb** repository
5. Select **Existing Azure Pipelines YAML File**
6. Select the **main** branch and the **/.ado/eshoponweb-ci.yml** file
7. Select **Continue**
8. Select the **Run** button to run the pipeline
9. Your pipeline will take a name based on the project name. Let's **rename** it for better identification
10. Go to **Pipelines > Pipelines** and select the recently created pipeline
11. Select on the ellipsis and **Rename/Remove** option
12. Name it **eshoponweb-ci** and select **Save**

Import and run the CD pipeline

In order to determine an **Azure** region with capacity, select a region from the dropdown below and then select the **Check Selected Region** button. Once a region is found to have capacity, you may proceed.

eastuseastus2westuswestus2westus3westeuropecanadacentralsoutheastasiaaustrali
aeast

Success

Region has capacity: westeurope

Let's import the CD pipeline named [eshoponweb-cd-webapp-code.yml](#).

1. Go to **Pipelines > Pipelines**
2. Select **New pipeline**
3. Select **Azure Repos Git (Yaml)**
4. Select the **eShopOnWeb** repository
5. Select **Existing Azure Pipelines YAML File**
6. Select the **main** branch and the **/.ado/eshoponweb-cd-webapp-code.yml** file
7. Select **Continue**
8. In the YAML pipeline definition, update the **variable** section:
 - **resource-group:** AZ400-RG1
 - **location:** westeurope
 - **templateFile:** webapp.bicep
 - **subscriptionid:** a02b18fa-a3f5-4d1e-8295-680337f124f3
 - **azureserviceconnection:** azure subs
 - **webappname:** az400-webapp-61807925

In this lab, please skip steps 9 and 10.

9. In the same YAML pipeline definition, notice the **task:**
AzureResourceManagerTemplateDeployment@3 section,
inputs values:
 - **csmFile:** \\$(Pipeline.Workspace)/eshoponweb-ci/Bicep/\$(templateFile)

- this points to the webapp.bicep file in the /Bicep folder in the repo; it's a required setting for the AzureResourceManagerTemplateDeployment task configuration.

10. In the same YAML pipeline definition, notice the **task:**

AzureRMWebAppDeployment@4 section,
inputs values:

- **packageForLinux:** `\$(Pipeline.Workspace)/eshoponweb-ci/WebSite/Web.zip`
- this refers to the Webdeploy zip package (artifact), created out of the Continuous Integration pipeline.

11. Select **Save and Run**

12. Open the pipeline and wait for it to execute successfully. Deployment should take about 3-5 min on average.

Important: If you see the message "This pipeline needs permission to access resources before this run can continue", select on View, Permit and Permit again.

13. Rename the pipeline to **eshoponweb-cd-webapp-code** for better identification

14. Navigate to the Azure Portal - App Services and select your deployed App Service. From the **Overview** tab, click on the URL in **Default domain** to open the link in your browser.

15. Confirm the EShopOnWeb web app is running as expected, and showing several product items.

Note: The first time you load the web app, or after a restart of the App Service, it will take a minute or more (browser showing Loading... message in the browser tab), before the site is fully loaded and ready. This is expected, since the product database is getting seeded and loaded into memory on the first load.

Create Azure App Configuration service

You'll create the Azure App Configuration service to centrally store the application configuration and feature flags.

1. In the Azure portal, search for **App Configuration** and select **Create app configuration**
2. Select the same Resource Group you used for the App Service deployment earlier

3. Specify the same location you used for the App Service deployment for the app configuration resource
4. Enter name: **appcs-61807925**
5. Select the **Standard** pricing tier for this lab (required for feature flags)
6. Click **Next: Access settings** and select **Enable Access Keys** under Authentication type
7. Select **Pass-through (Recommended)** as Authentication Method

Note: By enabling both the key-based Authentication type and Pass-through Authentication method, you as an admin get immediate access to configure actual App Configuration settings, as well as enabling Azure RBAC permissions for the Web App resource. Without enabling access keys, there is a potential waiting time of 15min before you as an admin can define App Configuration settings.

1. Select **Review + create** followed by **Create** and wait for the resource deployment to complete.
2. Once the resource is created, go to the resource

Set up feature flags in App Configuration

You'll create a feature flag to control application features dynamically. In this example, we show a "SalesWeekend" banner on the home page. The Feature Flag toggle enables or disables this.

1. In the left pane of the App Configuration service, select **Feature manager**
2. Select **Create:**
 - **What will you be using your feature flag for:** Switch
 - **Enable feature flag:** toggle to enable
 - **Feature flag name:** SalesWeekend
 - **Key:** *.appconfig.featureflag/SalesWeekend* (Gets filled automatically)
 - **Label:** leave empty
 - **Description:** Enables the SalesWeekend promotion banner
3. Confirm the creation with **Review + Create** and once more **Create**

Configure the Web Application to use App Configuration, using Managed Identity RBAC

1. In the Azure Portal, App Services, go to the WebApp you deployed earlier
2. From **Settings / Identity, System Assigned** tab, click the **Status** toggle to **On**
3. Click **Save** to save the changes
4. Confirm the popup message *enable system assigned managed identity* with **Yes**
5. Wait for the **Object (principal) ID** to get created
6. Navigate to the **App Configuration** resource, **Access Control (IAM)** tab
7. Click **Add+** / Add Role Assignment
8. In the *Search by role name, description, permission, or ID*, field, search for **App Configuration Data Reader** and select it
9. in the **Add Role Assignment** page / **Members** tab, **Assign Access To**, select **Managed Identity**
10. click the **+ Select Members** link, which opens the **Select Managed Identities** blade
11. Under **Managed Identity**, select **App Service (x)**, and select your **App Service Identity**
12. Confirm by clicking **Select**
13. Confirm by clicking **Review + Assign** twice
14. You can **validate the RBAC permission**, by navigating back to the **Access Control (IAM)** tab of the **App Configuration** resource, select **Role Assignments** and search/filter on **App Configuration**. This will show the App Configuration Data Reader role, and your App Service Managed Identity

Add App Configuration Environment Variables

In this step, you'll define several App Service Environment Variables to connect to Azure App Configuration.

1. In the Azure Portal, go to your deployed **App Services Web App**
2. Navigate to **Settings / Environment Variables**
3. Notice a few Variables are already defined; don't make any changes to the values or parameters

4. Click **+ Add**, to create the following 2 new variables:

Note: use the "Show Values" option (the eye icon) to unhide the characters while typing

- **Name:** AppConfigEndPoint
- **Value:** The URL of the App Configuration resource, including **https://** (_https://%yourappconfigname%.azconfig.io)
- **Name:** UseAppConfig
- **Value:** true

1. Click **Apply**, **Apply** and **Confirm** to save the changes

Test feature flag changes

1. In your web application, observe whether the ""SalesWeekend" banner (*All T-Shirts on sale this weekend*) feature is displayed
2. Go back to App Configuration in the Azure portal
3. In **Feature manager**, find the **SalesWeekend** feature flag
4. Toggle its state (enable/disable)
5. Select **Apply**
6. After about 10 seconds, refresh your web application
7. Notice that the "SalesWeekend" banner appears or disappears based on the flag state

Set up dynamic configuration parameters using Configuration Explorer in App Configuration

You'll create a dynamic configuration parameter to control application features dynamically. In this example, we show a "different message to the user" for empty search results, instead of what has been hard-coded in the application codebase.

1. From the **Web App Home Page**, notice the **Brand** and **Type** filter options.
2. In **Brand**, select **.NET**; in **Type**, select **USB Memory Stick**, and press the arrow key to execute a search
3. Notice the default response message "THERE ARE NO RESULTS THAT MATCH YOUR SEARCH"

Using App Configuration, we will change this message, without needing to make changes to the application code.

1. In the left pane of the App Configuration service, select **Configuration Explorer**
2. Select **Create**
3. Select **Key-Value**
 - **Key:** eShopWeb:Settings:NoResultsMessage
 - **Value:** Sorry, we couldn't find what you're looking for. But hey, why not checking for a different item. We heard the T-Shirts are sway!
 - **Label:** leave empty
 - **Content Type:** leave empty
4. Confirm the creation with **Apply** and once more **Create**
5. From the **Web App Home Page**, notice the **Brand** and **Type** filter options.
6. In **Brand**, select **.NET**; in **Type**, select **Mugs**. Which will show a few product items.
7. Now, select **Brand .NET** and **Type USB Memory Stick**.
8. Notice the **new custom message** as defined in the App Configuration Explorer

Note: This works because the app code itself looks for a parameter *eShopWeb:Settings:NoResultsMessage* and uses the value from App Config.

Clean up resources

Remember to delete the resources created in the Azure portal to avoid unnecessary charges:

1. Delete the resource group containing your App Configuration and Web App resources
2. In the Azure portal, navigate to your resource group
3. Select **Delete resource group**
4. Type the resource group name to confirm deletion
5. Select **Delete**

Summary

In this lab, you learned how to:

- **Enable dynamic configuration** using Azure App Configuration

- **Manage feature flags** for controlled feature rollouts
- **Configure applications** to consume centralized configuration
- **Test configuration changes** without application redeployment

Azure App Configuration provides a powerful way to manage application settings and feature flags centrally, enabling dynamic configuration changes and controlled feature rollouts without requiring application redeployment. This leads to more flexible and maintainable cloud applications.